Configuración de un clúster Kubernetes para SARSEG

Grafana con Deployment y ReplicaSet

Actividad Configuración de cluster Docker Swarm o Kubernetes · 15 por ciento P2
Autor del repositorio JOURT1 · 11/11/2026 · Zona horaria America/Guayaquil

Se desplegó Grafana como plataforma de observabilidad para un laboratorio vinculado con SARSEG. Se verificaron dos réplicas distribuidas en dos workers, exposición por NodePort, escalado, reposición automática de un Pod, actualización gradual y rollback. Los resultados se sustentan en estados de Kubernetes, solicitudes HTTP y capturas reales exportadas del tablero.

Repositorio público
https://github.com/JOURT1/SARSEG-Kubernetes-P2

Alcance y relación con SARSEG

SARSEG es un sistema de registro de asistencia con API .NET, PostgreSQL e interfaz Ionic/Angular. Grafana permite observar infraestructura y, con instrumentación posterior, podría mostrar latencia de la API, errores y trabajos de reportes. Para esta actividad no se necesita copiar el código fuente de la aplicación: se utilizan imágenes oficiales de la plataforma de monitoreo.

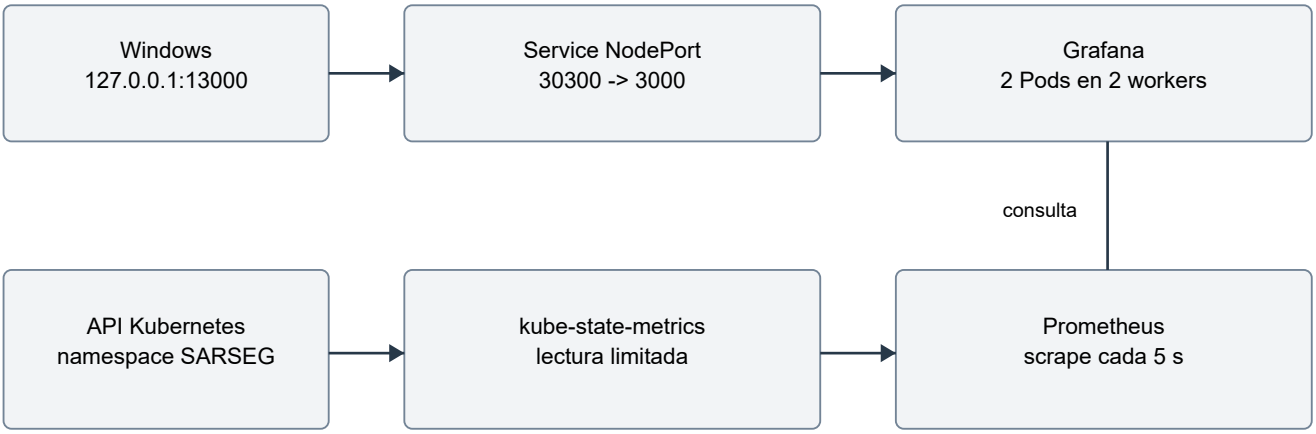
La práctica mide recursos reales de su namespace de observabilidad. No despliega la API privada de SARSEG ni presenta métricas de estudiantes o de la base de datos del sistema. El repositorio original conserva su visibilidad privada.

Resultado verificado

Criterio	Resultado
Clúster	1 control plane y 2 workers Ready; Kubernetes 1.32.2
Plataforma	Grafana 12.3.0 con Deployment y dos Pods Ready
Métricas	Prometheus 3.5.0 y kube-state-metrics 2.16.0; tres fuentes UP
Exposición	http://127.0.0.1:13000 mediante NodePort 30300
Pruebas finales	11 comprobaciones aprobadas; 113/113 muestras de salud correctas

El laboratorio corre sobre Docker Desktop con kind. Las réplicas demuestran mecanismos de Kubernetes en una computadora; no constituyen una instalación de alta disponibilidad de producción.

Arquitectura y configuración declarativa



Deployment ReplicaSet y Pod

El Deployment declara el estado deseado y gestiona las revisiones de la plantilla. Un ReplicaSet mantiene el número de Pods de una plantilla concreta. El Pod es la unidad que ejecuta el contenedor. La relación se comprobó por ownerReferences: Deployment → ReplicaSet → Pods. No se creó un ReplicaSet independiente que compitiera por el mismo selector.

El Service selecciona app=grafana y dirige tráfico únicamente a endpoints listos. Las probes comprueban /api/health y la afinidad excluye el control plane del reparto entre workers. El escalado modifica replicas; cambiar spec.template provoca una nueva revisión y otro ReplicaSet.

Parámetro	Valor y finalidad
Réplicas y distribución	replicas: 2; maxSkew: 1 entre los workers
Actualización	RollingUpdate; maxSurge: 1; maxUnavailable: 0
Terminación	preStop: sleep 10; terminationGracePeriodSeconds: 30
Recursos por Pod Grafana	Solicitud 100m CPU y 160Mi; límite 500m CPU y 512Mi
Acceso y datos	Viewer anónimo local; dashboards y datasource desde ConfigMaps

```
spec:
  replicas: 2
  strategy:
    type: RollingUpdate
    rollingUpdate: {maxSurge: 1, maxUnavailable: 0}
  selector:
    matchLabels: {app: grafana}
```

Configuración completa y reproducible: k8s/grafana.yaml y cluster/kind.yaml.

Grafana expuesto y fuentes activas

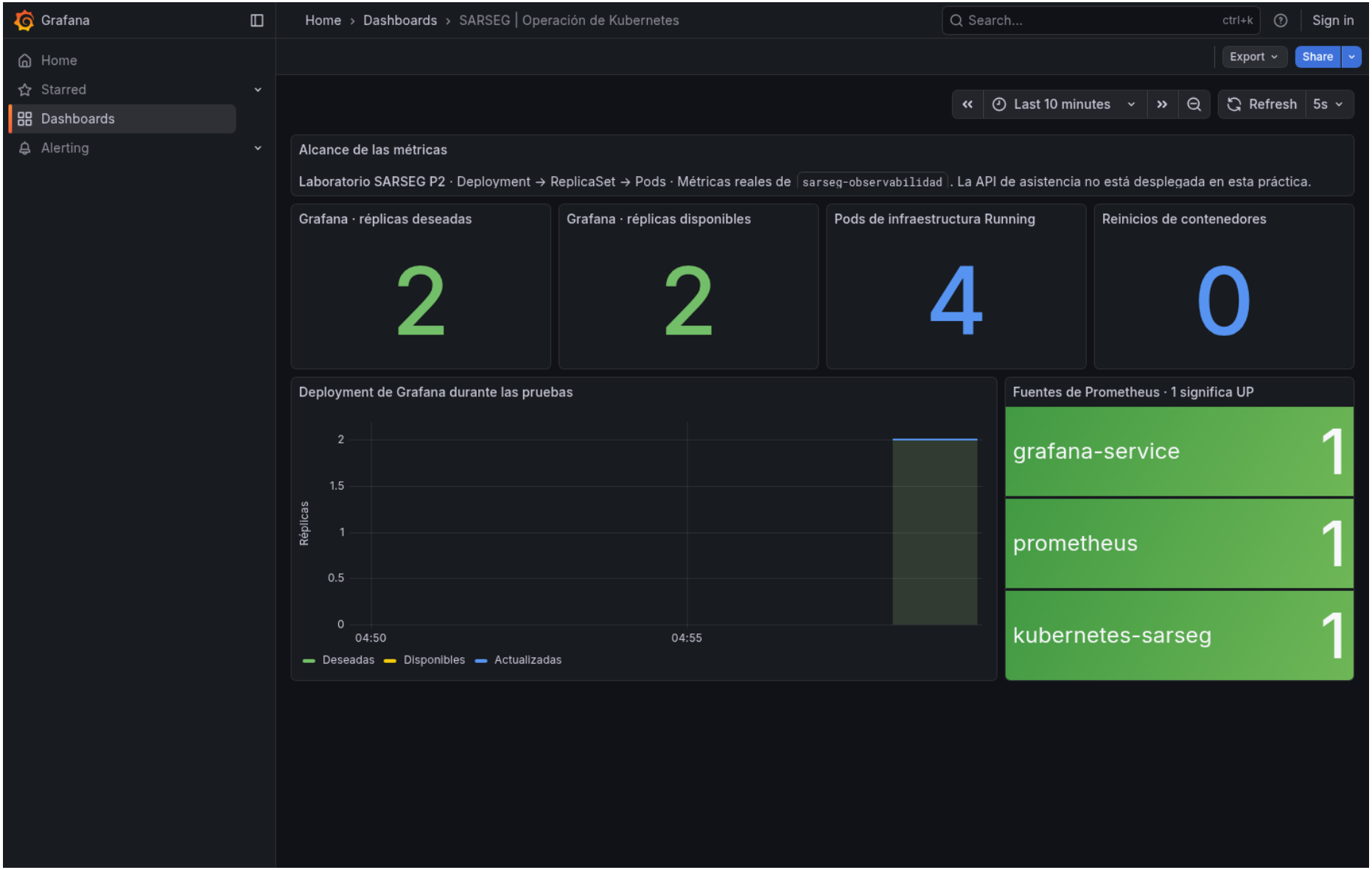


Figura 1. Exportación PNG real de Grafana Image Renderer antes del protocolo. Se observan dos réplicas deseadas y disponibles, cuatro Pods de infraestructura y tres fuentes UP. La exposición desde Windows se verificó además con HTTP 200 en /api/health. El PNG original está en docs/evidencias/capturas/01-grafana-base.png.

Escalado y recuperación de un Pod

Escalado observado

Se ejecutó kubectl scale deployment/grafana --replicas=3. Se esperaron tres Pods Ready y tres réplicas disponibles. El UID del ReplicaSet activo permaneció igual, lo que confirma que escalar no modifica la plantilla. Después se restauró el estado de dos réplicas.

```
$ctx = 'kind-sarseg-p2'
$ns = 'sarseg-observabilidad'
kubectl --context $ctx -n $ns scale deployment/grafana --replicas=3
# Despues de comprobar 3 replicas Ready
kubectl --context $ctx -n $ns scale deployment/grafana --replicas=2
```

Propiedad verificada en la ejecución final

Objeto	Nombre y referencia
Deployment	grafana; UID ad9753cd-55f6-4bb0-8bf2-d44f8db0b379
ReplicaSet activo	grafana-5c4c8dcb6c; owner UID del Deployment
Pod inicial 1	grafana-5c4c8dcb6c-6rchm; nodo sarseg-p2-worker
Pod inicial 2	grafana-5c4c8dcb6c-mpqjd; nodo sarseg-p2-worker2

Recuperación automática

Se eliminó el Pod **grafana-5c4c8dcb6c-6rchm** con kubectl delete pod --wait=false. El ReplicaSet creó un reemplazo y recuperó dos réplicas disponibles en aproximadamente **15.64 segundos**, según el tiempo medido por el script. La medición incluye el sondeo y el arranque; no es un SLA.

```
Pod eliminado
grafana-5c4c8dcb6c-6rchm
UID d03223cf-71e9-4cfd-aa8e-49277b23f0c2

Pod de reemplazo
grafana-5c4c8dcb6c-sclrn
UID 6fad7f7d-e630-480c-8b44-c3c10a0bea17
```

Los UIDs diferentes prueban que se creó otro objeto Pod. El criterio de aceptación exige Ready y availableReplicas, además del estado Running. Los archivos 02-escalado-tres y 03-recuperacion conservan las salidas y estados JSON.

Grafana escalado a tres réplicas

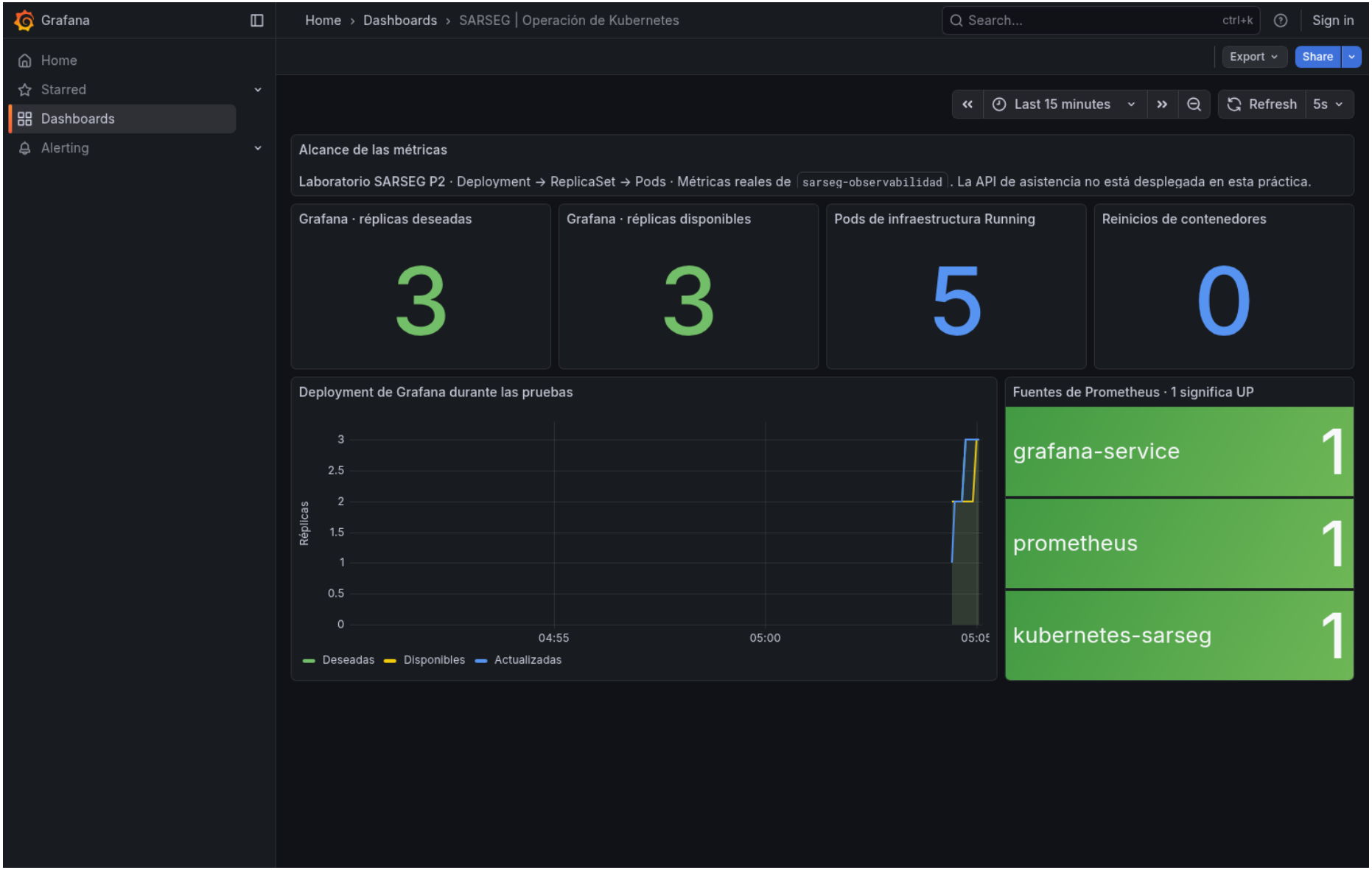


Figura 2. Captura exportada durante la ejecución final, una vez alcanzadas tres réplicas disponibles. Prometheus registra el cambio de estado real de Kubernetes. El incremento de Pods Running incluye los recursos del laboratorio; no representa estudiantes ni sesiones de asistencia.

Actualización gradual rollback y salud HTTP

La prueba cambió la anotación sarseg.io/revision-laboratorio de v1 a v2-prueba-p2 dentro de spec.template. La imagen continuó siendo grafana/grafana:12.3.0. Se demuestra el mecanismo de actualización de plantilla; no se afirma haber actualizado la versión del software.

Momento	ReplicaSet y resultado
Antes de actualizar	grafana-5c4c8dcb6c; dos réplicas disponibles
Después del rollout	grafana-557d55f7c; nuevo UID y dos réplicas disponibles
Rollback	grafana-5c4c8dcb6c; plantilla v1 y ReplicaSet original reactivado

```
$ctx = 'kind-sarseg-p2'
$ns = 'sarseg-observabilidad'
kubectl --context $ctx -n $ns rollout status deployment/grafana
kubectl --context $ctx -n $ns rollout undo deployment/grafana --to-revision=6
```

Hallazgo conservado y corrección

La primera ejecución detectó RemoteDisconnected en una solicitud durante el rollout y se marcó como no aprobada. El evento es compatible con una carrera entre la retirada del endpoint y la terminación del proceso. Se añadió un hook preStop de diez segundos para dar tiempo al drenaje y se repitió todo el protocolo. Se conserva la evidencia de ambas ejecuciones.

Ejecución	Muestras correctas	Resultado
Primera, sin preStop	110/111	Una conexión interrumpida
Final, con preStop	113/113	Todas las muestras HTTP 200 y database=ok

El muestreo se realiza aproximadamente cada segundo durante escalado, eliminación de Pod, rollout y rollback. Permite afirmar que esas solicitudes respondieron correctamente; no demuestra ausencia de interrupciones entre muestras ni disponibilidad de producción.

Evidencias: resultados.json, http-muestreo.json, historial-rollout.txt y estados 04, 05 y 06 de docs/evidencias/ejecucion-final. Los ReplicaSets antiguos con cero réplicas se conservan para el historial y el rollback.

Estado final después del rollback

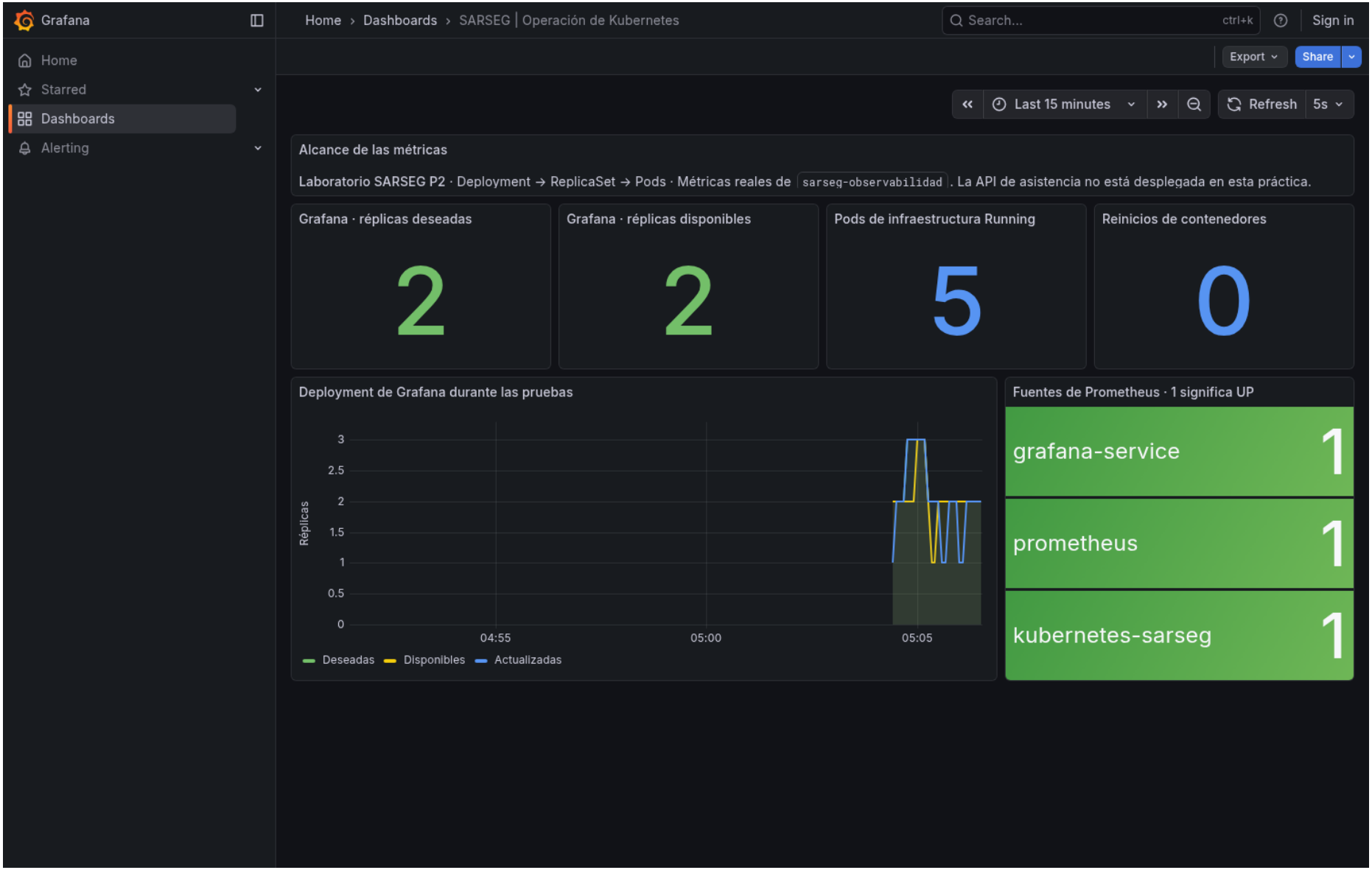


Figura 3. Captura exportada al finalizar el protocolo: dos réplicas deseadas y disponibles, fuentes UP e historial de cambios durante las pruebas. Los picos transitorios de Pods pueden incluir objetos en terminación. La disponibilidad se valida con Ready y availableReplicas, no solo con el contador Running.

Historial de trabajo y reproducción

Los avances se publicaron en ramas separadas con commits de alcance concreto. Se conservaron los cambios correctivos y el primer resultado fallido. Los pull requests usan merge commits para mantener visible el desarrollo; no se simulan revisores ni fechas de trabajo.

Rama	Avance y pull request
main	Alcance inicial y recepción de los avances integrados
codex/infraestructura-kubernetes	Clúster, manifiestos y arranque; PR 1
codex/metricas-sarseg	Métricas, ConfigMaps, tablero y afinidad; PR 2
codex/pruebas-deployment-replicaset	Pruebas, corrección de drenaje y evidencias; PR 3
codex/informe-evidencias	Informe, referencias e índice de entrega; PR 4

Trazabilidad: PR 1 · PR 2 · PR 3 · PR 4 · Historial de commits · Ramas.

Verificación independiente en GitHub Actions

El workflow creó otro clúster en un runner Linux, desplegó los manifiestos y ejecutó el protocolo real sin renderer. La ejecución terminó correctamente: 42963098362. Los registros están disponibles como artefacto de la ejecución.

Inicio desde una copia nueva

```
git clone https://github.com/JOURT1/SARSEG-Kubernetes-P2.git
cd SARSEG-Kubernetes-P2
.\scripts\Start-Lab.ps1

# Abrir http://127.0.0.1:13000
# Prueba nueva sin sobrescribir la evidencia entregada
python scripts/test_lab.py --output tmp/repeticion
```

Requisitos: Docker Desktop con motor Linux, PowerShell 7, kubectl 1.32 y Python 3 para las pruebas. Start-Lab.ps1 descarga kind 0.27.0 y valida el SHA256 oficial. Para capturas opcionales se ejecuta Export-Dashboard.ps1 y se agrega --render a las pruebas. Todos los comandos de Kubernetes especifican contexto y namespace.

Usos límites y conclusiones

Usos que puede tener la plataforma

En SARSEG, Grafana puede centralizar indicadores de operación y facilitar la detección de problemas durante horarios de entrada o salida. Al instrumentar la API .NET y sus dependencias se podrían añadir latencia HTTP, errores de autenticación agregados, tiempos de generación de PDF o XLSX y fallos de conexión con PostgreSQL. Los datos personales y las marcas individuales no son necesarios para esos paneles.

Deployment permite repetir la configuración, escalar la capa de visualización y actualizar su plantilla con control de réplicas disponibles. ReplicaSet repone Pods eliminados o perdidos. Service mantiene un punto estable de acceso aunque cambien las IP y los nombres de los Pods. Estas funciones se comprobaron con cambios reales y referencias de propiedad.

Límites del resultado

El clúster depende de una sola computadora y de Docker Desktop. Grafana usa SQLite efímera e independiente por réplica; los dashboards y la fuente de datos se reconstruyen desde ConfigMaps. Prometheus tiene una sola réplica y datos efímeros. No se habilitan edición de tableros ni alertas persistentes. Una solución productiva de Grafana en alta disponibilidad requiere una base compartida PostgreSQL o MySQL, autenticación y diseño de persistencia y redundancia.

El acceso anónimo tiene rol Viewer y se publica únicamente en 127.0.0.1. kube-state-metrics dispone de lectura limitada al namespace; se verificó que no puede listar Secrets. No se despliega ni modifica el sistema privado de asistencia. Las métricas implementadas son de infraestructura del laboratorio.

Conclusión

La ejecución final cumplió 11 comprobaciones: clúster Ready, propiedad de recursos, exposición, métricas, RBAC, escalado, reposición de Pod, rollout, rollback y salud HTTP. Se recuperaron dos réplicas después de los cambios y se observaron 113 respuestas correctas de 113 muestras. El hallazgo de la primera ejecución y su corrección muestran el proceso de validación y sus límites.

Fuentes y archivos de respaldo

Kubernetes Deployments

Kubernetes ReplicaSet

kind Configuration

Grafana Provisioning

Grafana High availability

Grafana Image rendering

Material de clase consultado: Intro Kubernetes, páginas 6 a 10 y 13; Intro Docker, página 9; CD Intro, páginas 3 y 6; Intro Entrega Continua CD, página 4. El ejemplo de primer Pod usa Nginx, mientras esta práctica despliega Grafana. Los documentos de clase no se redistribuyen en el repositorio.

Respaldo verificable en docs/evidencias: ejecucion-final/resultados.json, estados JSON y TXT, comandos.json, eventos.txt, http-muestreo.json, capturas PNG y SHA256SUMS.txt. La ejecución inicial se conserva en ejecucion. Las imágenes son exportaciones reales de Grafana Image Renderer; las salidas de kubectl se identifican como registros de terminal.